

PYTHON FOUNDATIONS

5 Hands-On Projects to Master Core Programming Concepts

Welcome to Your Python Practice Journey! This document contains 5 tailored practice projects designed to take you from a complete beginner to a confident intermediate programmer. Instead of looking at complete code solutions, you will build these programs yourself using detailed real-world logic, specific input patterns, and strict test cases. No python code is provided inside this document, ensuring you practice the syntax completely from scratch.

Project 1: The Daily Pocket Money & Expense Tracker

Difficulty Level: Very Basic (Beginner First Step)

Concepts Used: Variables Data Types (Float, String) Arithmetic Operators If-Else Conditions
String Concatenation

Real-World Analogy

Imagine you have a physical piggy bank where you keep your daily allowance. Every time you buy a snack, a notebook, or a bus ticket, you write down the cost on a sticky note. At the end of the day, you count how much money is left and check if you spent more than your goal limit so you don't run out of cash.

Detailed Project Description

You need to write a simple terminal program that acts as a financial health checker for a single day. The program will first greet you by your name, ask for your total daily pocket money budget, and then ask for two specific expenses you made today. It will calculate the total money spent, the remaining money left in your wallet, and then evaluate your saving habits using simple conditional checks.

What the Program Should Ask For (Inputs)

1. **User's Name:** A text string input.
2. **Daily Budget Amount:** A decimal number representing how much money you can spend.
3. **Expense 1 Name & Cost:** A text name for the first item, and a decimal number for its price.
4. **Expense 2 Name & Cost:** A text name for the second item, and a decimal number for its price.

Expected Logic & Behavior

- Welcome the user by including their name in a custom greeting message.
- Add the two expense costs together using the addition operator to find the total spending.
- Subtract the total spending from the daily budget using the subtraction operator to find the balance.

- Check if the total spending is strictly greater than the daily budget. If it is, show a warning message.
- Check if the remaining balance is exactly half or more than half of the initial budget. If yes, print a celebratory reward message for saving well. Otherwise, print a normal summary.

EXAMPLE TEST CASE 1: OVER-SPENDING SITUATION

Inputs provided by user: Name = "Rahul", Budget = 100.0, Expense 1 = "Burger" (cost 65.5), Expense 2 = "Cold Drink" (cost 40.0)

```
Expected Output in Terminal:  
Hello Rahul! Let's calculate your daily spending structure.  
Total Money Spent: 105.5  
Remaining Balance: -5.5  
ALERT: Rahul, you have overspent your daily budget by 5.5 units! Please control  
your expenses.
```

EXAMPLE TEST CASE 2: GOOD SAVING SITUATION

Inputs provided by user: Name = "Sneha", Budget = 200.0, Expense 1 = "Pen" (cost 15.0), Expense 2 = "Notebook" (cost 45.0)

```
Expected Output in Terminal:  
Hello Sneha! Let's calculate your daily spending structure.  
Total Money Spent: 60.0  
Remaining Balance: 140.0  
GREAT JOB! Sneha, you saved more than half of your budget today! Keep it up.
```

Project 2: The Smart Grocery Shopping Assistant

Difficulty Level: Basic (Introduction to Collections)

Concepts Used: [Lists](#) [For Loops](#) [List Indexing](#) [Comparison Operators](#) [String Methods](#)

Real-World Analogy

Think of walking into a supermarket with a piece of paper listing items to buy. As you walk down the aisles, you check each item one by one. If an item matches a specific keyword or if it's the very first priority item on your list, you pay special attention to it. At the end, you count the items to ensure you didn't forget anything.

Detailed Project Description

This program manages an exact list of items to buy. You will pre-populate a list with 5 grocery items. The program will print out the single most important item (which always lives at the very first position of your list). Then, it will use a loop to inspect every item one by one. If an item name contains more than 5 characters, it

will mark it as a "Bulk Box Item". Finally, it will search for a specific item requested by the user to see if it exists on our shopping list.

What the Program Should Ask For (Inputs)

1. The program starts with a predefined list of 5 strings: `["Milk", "Chocolate", "Bread", "Apples", "Tomatoes"]`
2. **Search Query:** A text string entered by the user to find a specific item on the list.

Expected Logic & Behavior

- Access and display the item at position zero (Index 0) with a message saying "Priority Item: ".
- Run a loop that travels through the list from the beginning to the end.
- For each item inside the loop, check the length of the string. If the character length is greater than 5, print the item name followed by "(Requires Bulk Cart)". If it is 5 characters or less, print the item name followed by "(Standard Basket)".
- Take the search string input from the user. Convert it to lowercase to avoid matching errors, and use standard membership checks to tell the user if the item is present or missing.

EXAMPLE TEST CASE: SUCCESSFUL EXECUTION

Initial List: `["Milk", "Chocolate", "Bread", "Apples", "Tomatoes"]`

User Inputs: Search Item = "apples"

Expected Output in Terminal:

```
--- Grocery Assistant Dashboard ---  
Your Number 1 Priority Item is: Milk
```

Analyzing your items:

1. Milk -> (Standard Basket)
2. Chocolate -> (Requires Bulk Cart)
3. Bread -> (Standard Basket)
4. Apples -> (Standard Basket)
5. Tomatoes -> (Requires Bulk Cart)

Enter an item to search: apples

Result: Yes, 'apples' is already on your shopping list!

Project 3: School Report Card & Grade Analytics System

Difficulty Level: Basic-Intermediate (Structured Data)

Concepts Used: Dictionaries Tuples Custom Functions For Loops (Key-Value iteration)

Logical Operators

Real-World Analogy

Imagine a school teacher sitting down with a traditional paper grading ledger. Each student is assigned a permanent, unchangeable identity card containing their Roll Number and full legal name. Next to that identity card, the teacher records their final numerical exam score. The teacher then creates a set rules to calculate the class average and find out who scored the highest mark.

Detailed Project Description

In this project, you will move away from loose variables and store your data cleanly. You will use a Python Dictionary where each key is an unchangeable Tuple containing fixed student metadata: `(Roll Number, Student Name)`. The matching value in the dictionary will be their exam score (out of 100). You must write separate, structured blocks of reusable code (Functions) to perform calculations like finding the average class score and printing the grade sheet.

What the Program Should Ask For (Inputs)

This program is data-driven. It initiates a dataset containing 4 students inside the main code block:

- Tuple `(101, "Amit")` with score `85`
- Tuple `(102, "Bina")` with score `92`
- Tuple `(103, "Chirag")` with score `68`
- Tuple `(104, "Divya")` with score `45`

Expected Logic & Functions to Build

- **Function 1: `calculate\_average(grade\_dict)`** - It should accept the student dictionary, loop through all the marks, calculate the mathematical average (sum divided by total count), and return the result.
- **Function 2: `get\_letter\_grade(score)`** - It should accept a single numerical score and return a letter grade string based on these thresholds:
 - 90 or above = "A"
 - 75 to 89 = "B"
 - 50 to 74 = "C"
 - Below 50 = "Fail"
- The main section of the code must call these functions, iterate through the keys and values to print a formatted layout, and output the total statistics.

EXAMPLE TEST CASE: FINAL GRADE SHEET DISPLAY

Input Dataset: Dictionary containing the 4 student tuples and marks listed above.

Expected Output in Terminal:

```
===== CLASS REPORT CARD =====  
Roll No: 101 | Name: Amit   | Score: 85 | Grade: B  
Roll No: 102 | Name: Bina   | Score: 92 | Grade: A  
Roll No: 103 | Name: Chirag  | Score: 68 | Grade: C  
Roll No: 104 | Name: Divya   | Score: 45 | Grade: Fail  
-----  
Class Performance Metrics:  
Total Students Evaluated: 4  
Class Average Score: 72.5  
Highest Score in Class: 92 (Achieved by Bina)
```

Project 4: The Neighborhood Digital Library Archive

Difficulty Level: Intermediate (Advanced Manipulation)

Concepts Used: Lists inside Dictionaries List Slicing [:] String Extraction

Functions with Arguments & Returns While Loops

Real-World Analogy

Think of a multi-shelf library cabinet. Each shelf represents a specific genre of literature like "Sci-Fi" or "History". On each shelf, books are stored in a strict row from left to right. When a patron comes in, they can browse all books, ask to see a "slice" of books (like looking at just the first two books on the shelf), or add a brand-new book at the end of the shelf row.

Detailed Project Description

You will create an interactive library management terminal application. The database is represented by a dictionary where keys are genre names (strings), and values are Python Lists containing names of individual books. The program will offer a simple terminal menu that keeps running inside a loop until the user explicitly chooses to exit.

What the Program Should Ask For (Inputs)

1. **Menu Choice:** An integer input (1 to 4) to pick an action.
2. **Genre Entry:** A string input specifying which shelf to modify/view.
3. **Book Title:** A string input specifying the name of the book to add.

Expected Logic & Functions to Build

- **Initial Database Layout:** A dictionary initialized with:
 - `"Sci-Fi" → ["Dune", "Matrix", "Avatar", "Inception"]`
 - `"History" → ["Sapiens", "Pyramids", "Rome"]`
- **Function:** `add_new_book(database, genre, book_title)` - Finds the correct genre key and appends the new title into its list. If the genre does not exist, it creates a new list for that genre.
- **Function:** `preview_shelf(database, genre)` - Uses **List Slicing** to extract and print only the *first 3 books* from that genre's list. It handles cases where a shelf has fewer than 3 books gracefully without crashing.
- A constant running loop controls the interactive user input system until option 4 is typed.

EXAMPLE TEST CASE: INTERACTIVE SLICING & APPENDING MENU

```
--- LIBRARY SYSTEM ACTIVE ---
1. View Top 3 Books on a Shelf (Slicing)
2. Add New Book to a Shelf
3. Exit

Select Option: 1
Enter Genre Name: Sci-Fi
-> Previewing first 3 books on Sci-Fi Shelf: ['Dune', 'Matrix', 'Avatar']

Select Option: 2
Enter Genre Name: History
Enter Book Title: Vikings
-> Success! 'Vikings' has been logged.

Select Option: 1
Enter Genre Name: History
-> Previewing first 3 books on History Shelf: ['Sapiens', 'Pyramids', 'Rome']

Select Option: 3
Thank you for using the Archive. Goodbye!
```

Project 5: Personalized Multi-Day Travel Itinerary Planner

Difficulty Level: Intermediate (Full Concept Integration)

Concepts Used: **Nested Loops** **Complex Slicing & Reverse Indexing** **Dictionary of Tuples**

Functions with Multiple Return Values **String Formatting**

Real-World Analogy

Imagine planning an intensive roadmap vacation across different cities. For every single day of your trip, you map out a fixed path: a starting terminal code, a destination city name, and the estimated fuel or travel cost. To review your plans, you look at your map normally, check your final breakdown summary, or sometimes review your roadmap completely in reverse to plan your trip back home.

Detailed Project Description

This final project tests your mastery over every single structural element of Python. You will manage a complex dataset representing a 3-day travel itinerary. The core structure is a dictionary where each key is a string representing the day (e.g., "Day 1", "Day 2"). The value associated with each key is a Tuple holding exactly three distinct objects: `(String: Route Airport Code, String: Destination City, Integer: Estimated Cost)`.

What the Program Should Ask For (Inputs)

The system initializes with a pre-set data arrangement representing your complete travel path:

- `"Day 1" → ("DEL-MUM", "Mumbai", 4500)`
- `"Day 2" → ("MUM-GOA", "Goa", 3200)`
- `"Day 3" → ("GOA-BLR", "Bengaluru", 5100)`

1. **Budget Ceiling:** A maximum allowed expense cap integer specified by the user.
2. **Character Slicing Code:** A query asking the program to isolate and display only the airport destination short codes.

Expected Logic & Functions to Build

- **Function:** `analyze_trip(itinerary, max_budget)` - This function must read through your itinerary using an advanced loop. It extracts individual elements from inside the tuples via precise zero-based indexing. It must return two distinct values simultaneously: a total accumulated travel cost figure, and a new filtered list showing only the names of cities that cost less than your specified threshold.
- **String Slicing Task:** For each day, isolate and print out just the last 3 characters of the route code string (e.g., extracting `"MUM"` out of `"DEL-MUM"`) by utilizing negative step parameters or reverse slicing patterns.

EXAMPLE TEST CASE: COMPREHENSIVE EXECUTION PROFILE

Input Parameters: Initial Travel Itinerary Dictionary, Budget Ceiling = 5000

```
Processing your Complete Travel Route Map...

--- ROUTE ANALYSIS REPORT ---
Day 1: Flying to Mumbai. Airport Arrival Code: MUM
Day 2: Flying to Goa. Airport Arrival Code: GOA
Day 3: Flying to Bengaluru. Airport Arrival Code: BLR

Evaluating financial alerts with your maximum cap limit of 5000:
- Day 1 (Mumbai) matches your financial criteria. Price: 4500
- Day 2 (Goa) matches your financial criteria. Price: 3200
- WARNING: Day 3 to Bengaluru exceeds your filter limit! Costing: 5100

===== FINAL SUMMARY =====
Total Estimated Cost for the entire journey: 12800
Affordable Stops Found: ['Mumbai', 'Goa']
=====
```